ONLINE SALES

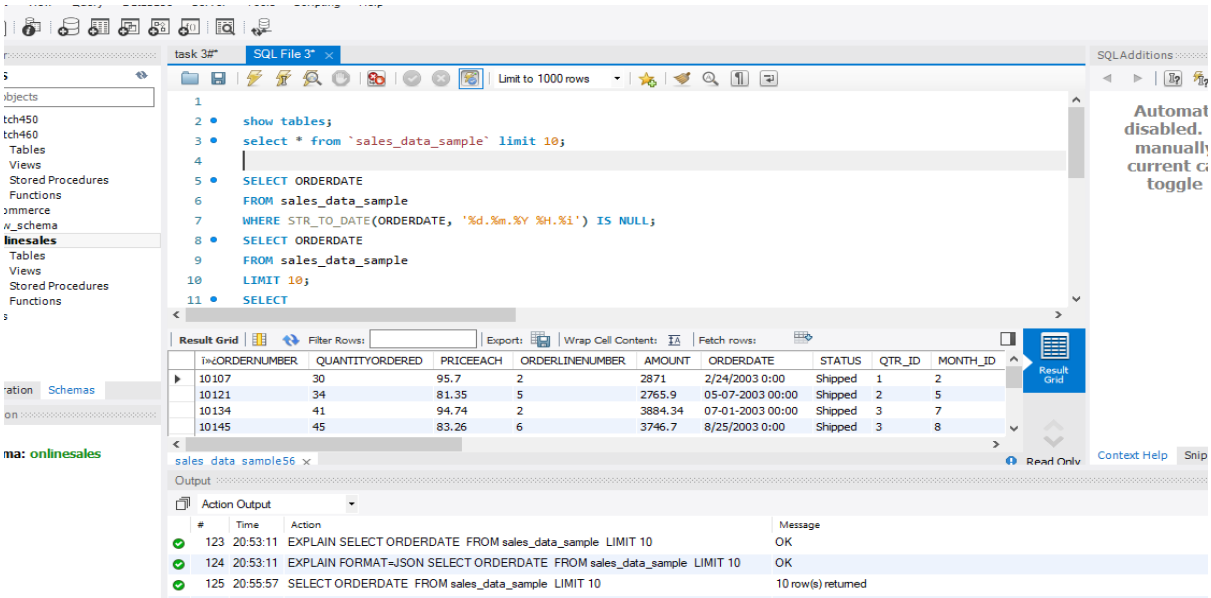
1. CREATE TABLE

- You're defining the structure of your dataset.
- Think of it like laying the foundation: every column has a role (ID, Date, Amount, Product).
- Without this, you can't store or query data.

2. INSERT DATA

- This is the **raw input** — the transactions.
- Each row is one order.
- Real-world: this would come from e-commerce logs, payment gateways, etc.

Data inserting



Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

ï»¿ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	AMOUNT	ORDERDATE	STATUS	QTR_ID	MONTH_ID	YE
10107	30	95.7	2	2871	2/24/2003 0:00	Shipped	1	2	20
10121	34	81.35	5	2765.9	05-07-2003 00:00	Shipped	2	5	20
10134	41	94.74	2	3884.34	07-01-2003 00:00	Shipped	3	7	20
10145	45	83.26	6	3746.7	8/25/2003 0:00	Shipped	3	8	20
10159	49	100	14	5205.27	10-10-2003 00:00	Shipped	4	10	20
10168	36	96.66	1	3479.76	10/28/2003 0:00	Shipped	4	10	20
10180	29	86.13	9	2497.77	11-11-2003 00:00	Shipped	4	11	20
10188	48	100	1	5512.32	11/18/2003 0:00	Shipped	4	11	20
10201	22	98.57	2	2168.54	12-01-2003 00:00	Shipped	4	12	20
10211	41	100	14	4708.44	1/15/2004 0:00	Shipped	1	1	20

3. MONTH ONLY EXTRACT

- Extracting the month number is a way to simplify time-based analysis.
- Instead of working with full dates, you reduce them to a manageable unit (Jan = 1, Feb = 2, etc.).
- Useful when you want to see seasonal trends

```
SELECT
  EXTRACT(MONTH FROM
    CASE
      WHEN ORDERDATE LIKE '%/%'
        THEN STR_TO_DATE(ORDERDATE, '%m/%d/%Y')
      WHEN ORDERDATE LIKE '%-%'
        THEN STR_TO_DATE(ORDERDATE, '%d-%m-%Y')
```

END

) AS month

FROM sales_data_sample;

```
LIMIT 10;
SELECT
  EXTRACT(MONTH FROM
    CASE
      WHEN ORDERDATE LIKE '%/%'
        THEN STR_TO_DATE(ORDERDATE, '%m/%d/%Y')
      WHEN ORDERDATE LIKE '%-%'
        THEN STR_TO_DATE(ORDERDATE, '%m-%d-%Y')
    END
  ) AS month
FROM sales_data_sample;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

month
2
5
7
8
10

Result 57

Full table

Result Grid | Filter Rows: | Export

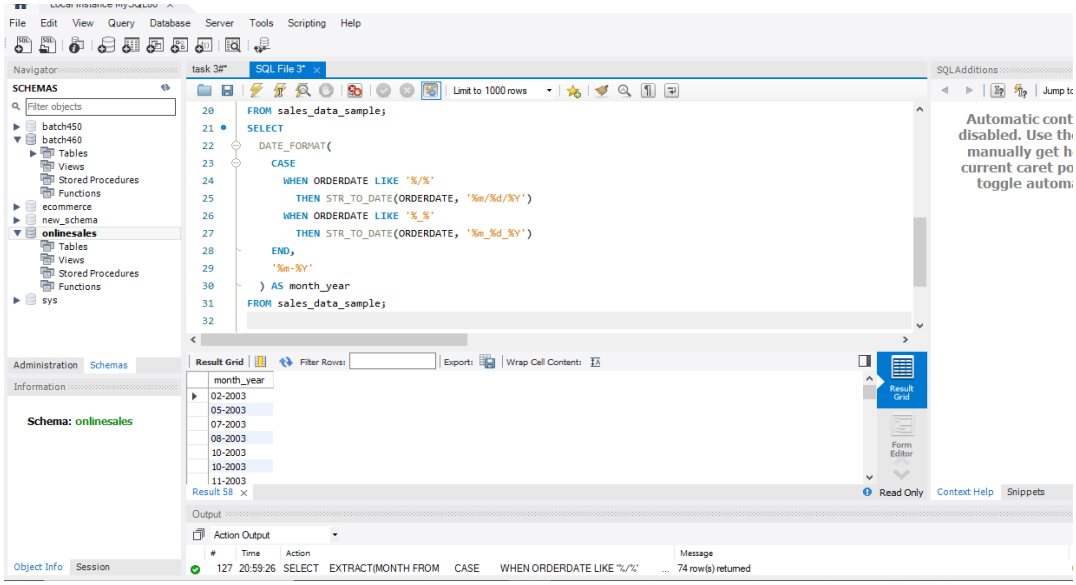
month
2
5
7
8
10
10
11
11
12
1
2
4

4. YEAR + MONTH GROUPING

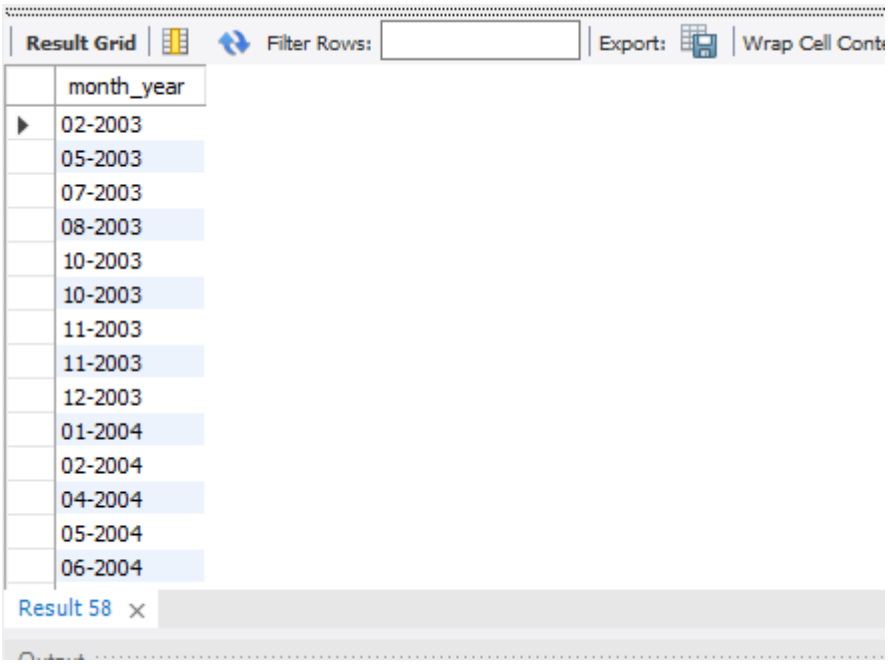
- Adds year context to month.
- Why? Because Jan 2024 ≠ Jan 2025.
- Grouping by both ensures you don't mix data across years .

```
SELECT
  EXTRACT(YEAR FROM
    CASE
      WHEN ORDERDATE LIKE '%/%' THEN STR_TO_DATE(ORDERDATE, '%m/%d/%Y')
      WHEN ORDERDATE LIKE '%-%' THEN STR_TO_DATE(ORDERDATE, '%d-%m-%Y')
    END
  ) AS year,
  EXTRACT(MONTH FROM
    CASE
      WHEN ORDERDATE LIKE '%/%' THEN STR_TO_DATE(ORDERDATE, '%m/%d/%Y')
      WHEN ORDERDATE LIKE '%-%' THEN STR_TO_DATE(ORDERDATE, '%d-%m-%Y')
```

```
END
) AS month
FROM sales_data_sample
GROUP BY year, month;
```



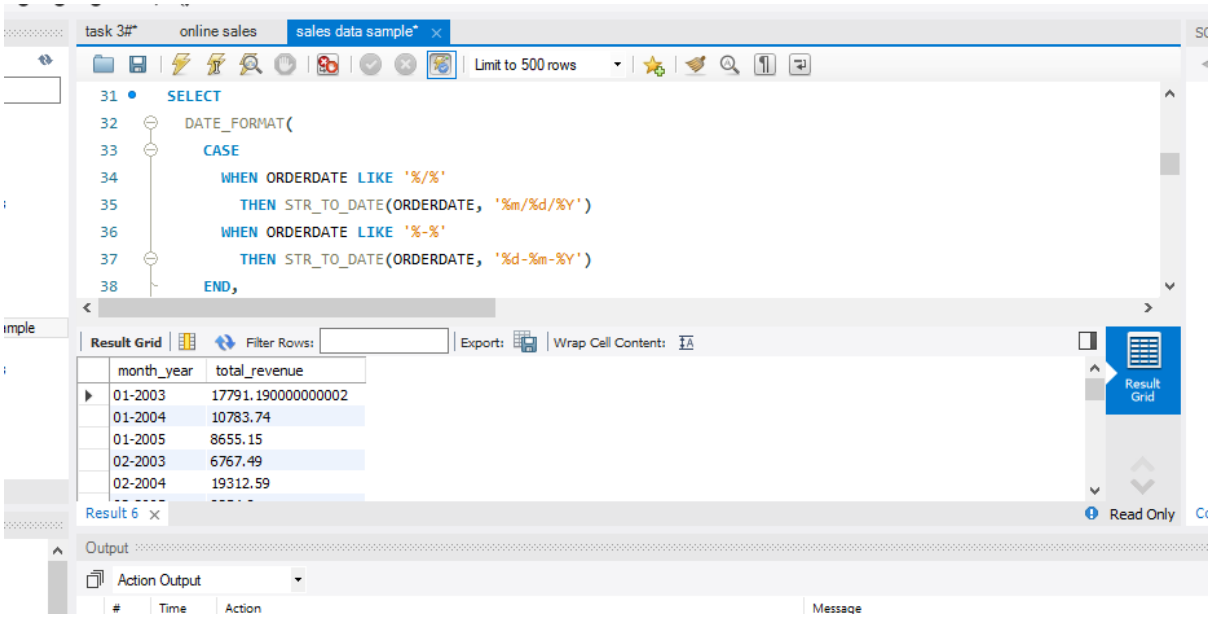
Full table



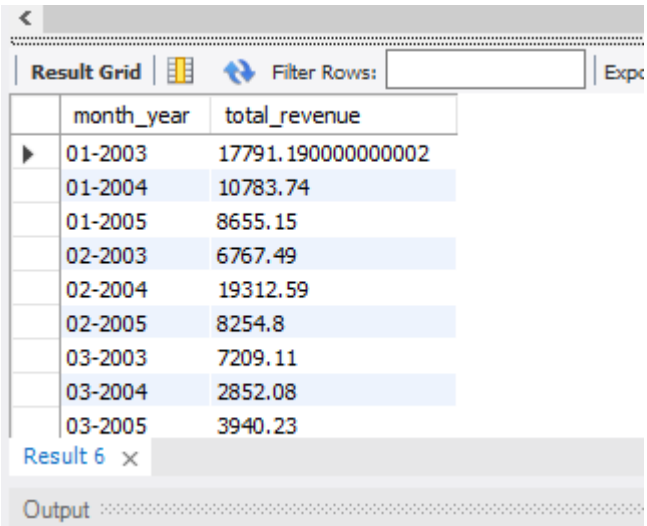
5. MONTHLY REVENUE (SUM)

- Revenue = total money earned.
- Summing by month gives you a financial snapshot.
- Example: CFOs look at this to track growth or decline.

```
SELECT
    EXTRACT(MONTH FROM
        CASE
            WHEN ORDERDATE LIKE '%/%' THEN
STR_TO_DATE(ORDERDATE, '%m/%d/%Y')
            WHEN ORDERDATE LIKE '%-%' THEN
STR_TO_DATE(ORDERDATE, '%d-%m-%Y')
        END
    ) AS month,
    SUM(amount) AS total_revenue
FROM sales_data_sample
GROUP BY month;
```



Full table



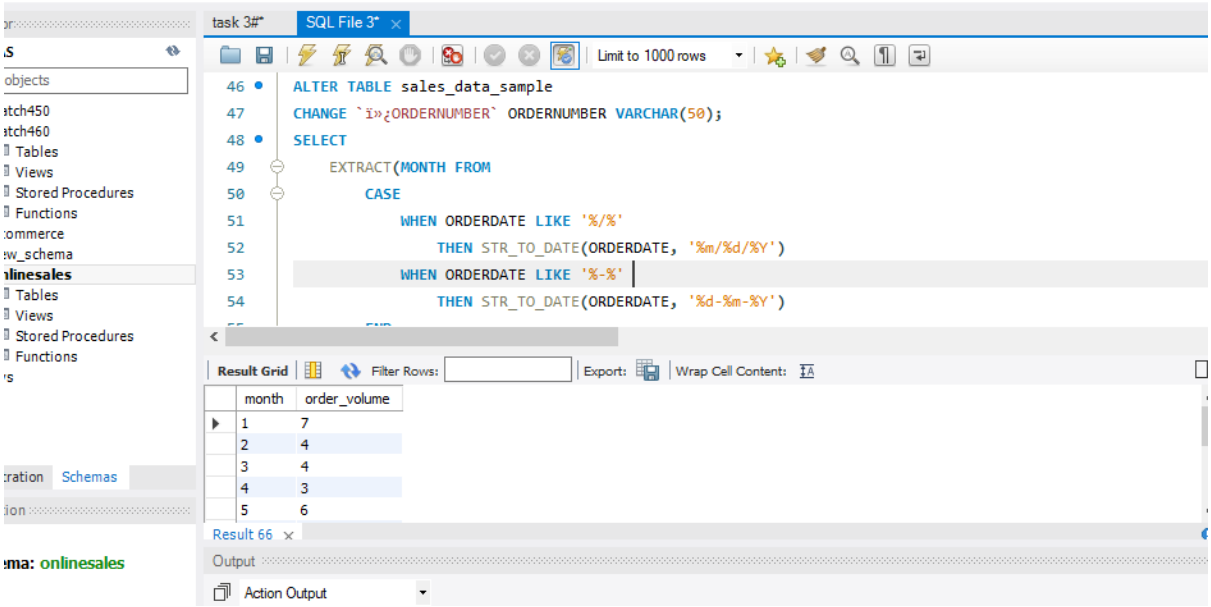
6. ORDER VOLUME (COUNT DISTINCT)

- Measures activity (how many orders).
- Even if revenue is high, order volume tells you if it’s from many small sales or few big ones.

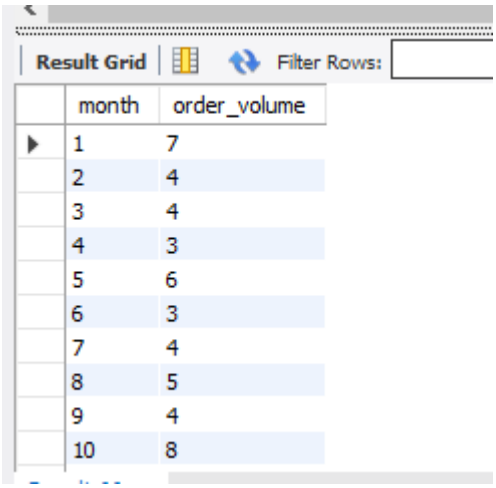
- This is critical for customer behavior analysis.

```
SELECT
  EXTRACT(MONTH FROM
    CASE
      WHEN ORDERDATE LIKE '%/%' THEN
STR_TO_DATE(ORDERDATE, '%m/%d/%Y')
      WHEN ORDERDATE LIKE '%-%' THEN
STR_TO_DATE(ORDERDATE, '%d-%m-%Y')
    END
  ) AS month,
  COUNT(DISTINCT order_id) AS order_volume
FROM sales_data_sample
GROUP BY month;
```

Order volume (count distinct);



Full table



month	order_volume
1	7
2	4
3	4
4	3
5	6
6	3
7	4
8	5
9	4
10	8

7. FULL MAIN ANALYSIS

- Combines revenue + volume in one view.
- This is the core business KPI query.
- You can see:
  - Which months are profitable

- Which months have high customer activity
- Whether revenue and volume move together or diverge

```
SELECT
  EXTRACT(YEAR FROM
    CASE
      WHEN ORDERDATE LIKE '%/%' THEN
STR_TO_DATE(ORDERDATE, '%m/%d/%Y')
      WHEN ORDERDATE LIKE '%-%' THEN
STR_TO_DATE(ORDERDATE, '%d-%m-%Y')
    END
  ) AS year,
  EXTRACT(MONTH FROM
    CASE
      WHEN ORDERDATE LIKE '%/%' THEN
STR_TO_DATE(ORDERDATE, '%m/%d/%Y')
      WHEN ORDERDATE LIKE '%-%' THEN
STR_TO_DATE(ORDERDATE, '%d-%m-%Y')
    END
  ) AS month,
  SUM(amount) AS total_revenue,
  COUNT(DISTINCT order_id) AS order_volume
```

FROM sales_data_sample

GROUP BY year, month

ORDER BY year, month;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

year	month	total_revenue	order_volume
2003	1	17791.19	3
2003	2	6767.49	1
2003	3	7209.11	1
2003	4	2793.86	1
2003	5	16393.95	2
2003	6	8014.82	1
2003	7	10140	2
2003	8	8939.65	1
2003	9	10993.5	1
2003	10	22223.03	3

Limit to 1000 rows

```
SELECT EXTRACT(YEAR FROM CASE WHEN ORDERDATE LIKE '%/%' THEN STR_TO_DATE(ORDERDATE, '%m/%d/%Y')
    WHEN ORDERDATE LIKE '%-%' THEN STR_TO_DATE(ORDERDATE, '%d-%m-%Y')END
    ) AS year,
    EXTRACT(MONTH FROM CASE WHEN ORDERDATE LIKE '%/%' THEN STR_TO_DATE(ORDERDATE, '%m/%d/%Y')
    WHEN ORDERDATE LIKE '%-%' THEN STR_TO_DATE(ORDERDATE, '%d-%m-%Y')END
    ) AS month, SUM(AMOUNT) AS total_revenue, COUNT(DISTINCT ORDERDATE) AS order_volume
FROM sales_data_sample GROUP BY year, month
ORDER BY year, month;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

year	month	total_revenue	order_volume
2003	1	17791.19	3
2003	2	6767.49	1
2003	3	7209.11	1
2003	4	2793.86	1
2003	5	16393.95	2

Result 68

Output

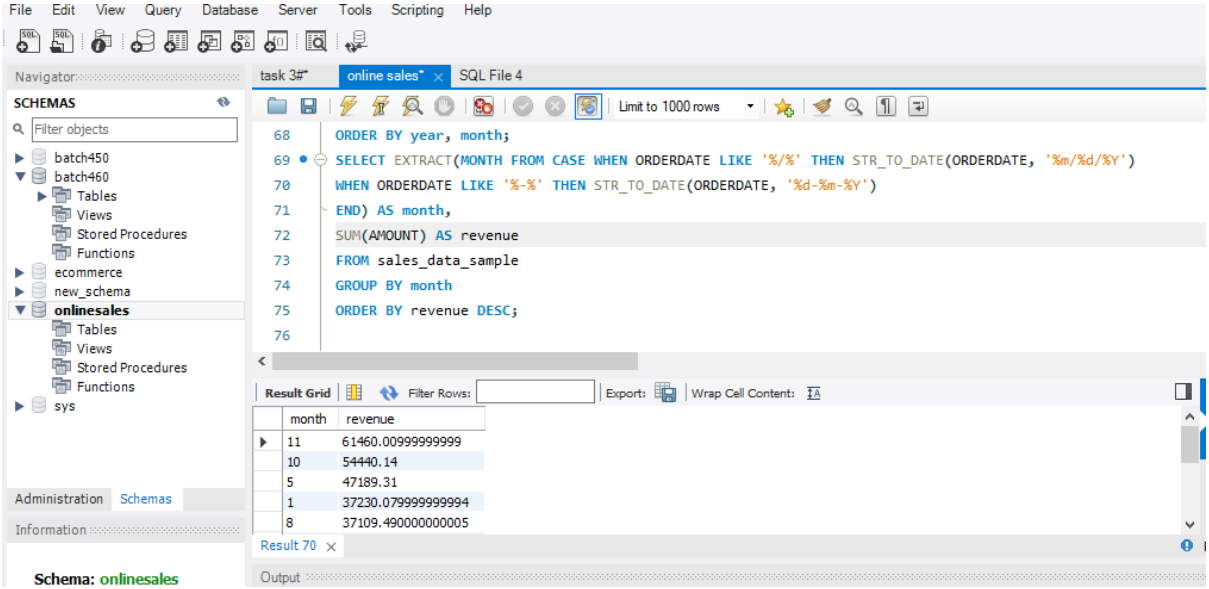
8. SORTING (ORDER BY)

- Ranking months by revenue.

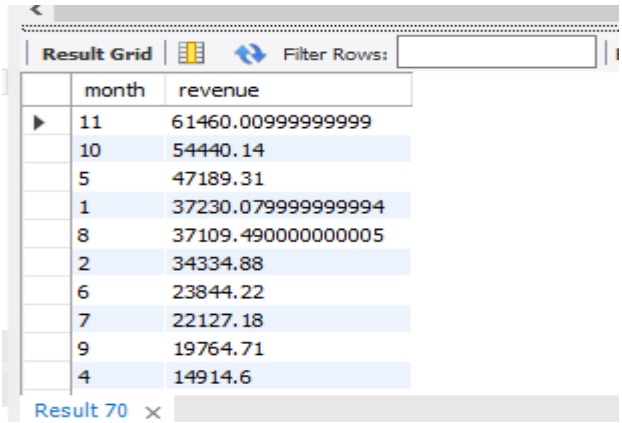
- Helps identify best-performing months.
- Example: April might be your peak season → plan marketing accordingly.

```
SELECT
  EXTRACT(MONTH FROM
    CASE
      WHEN ORDERDATE LIKE '%/%' THEN
STR_TO_DATE(ORDERDATE, '%m/%d/%Y')
      WHEN ORDERDATE LIKE '%-%' THEN
STR_TO_DATE(ORDERDATE, '%d-%m-%Y')
    END
  ) AS month,
  SUM(amount) AS revenue
FROM sales_data_sample
GROUP BY month
ORDER BY revenue DESC;
```

Sorting order by



Full table

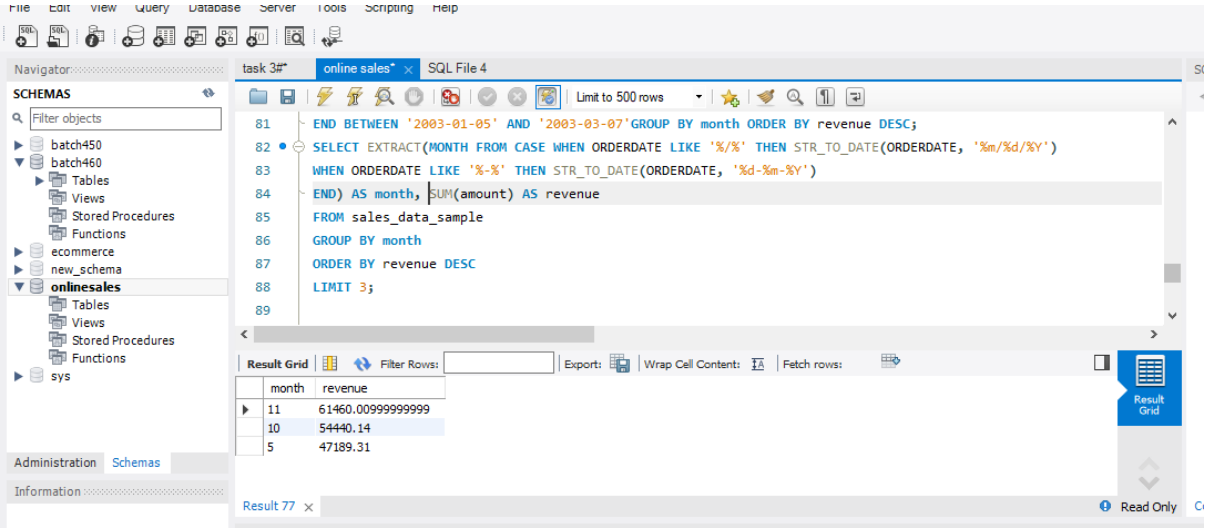


9. LIMIT (TOP 3 MONTHS)

- Focuses only on the top performers.
- Useful for executive dashboards where only highlights matter.
- Example: “Show me the top 3 months by sales.”

```
SELECT
    EXTRACT(MONTH FROM
```

```
        CASE
            WHEN ORDERDATE LIKE '%/%' THEN
STR_TO_DATE(ORDERDATE, '%m/%d/%Y')
            WHEN ORDERDATE LIKE '%-%' THEN
STR_TO_DATE(ORDERDATE, '%d-%m-%Y')
        END
    ) AS month,
    SUM(amount) AS revenue
FROM sales_data_sample
GROUP BY month
ORDER BY revenue DESC
LIMIT 3;
```



Full table

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

month	revenue
11	61460.00999999999
10	54440.14
5	47189.31

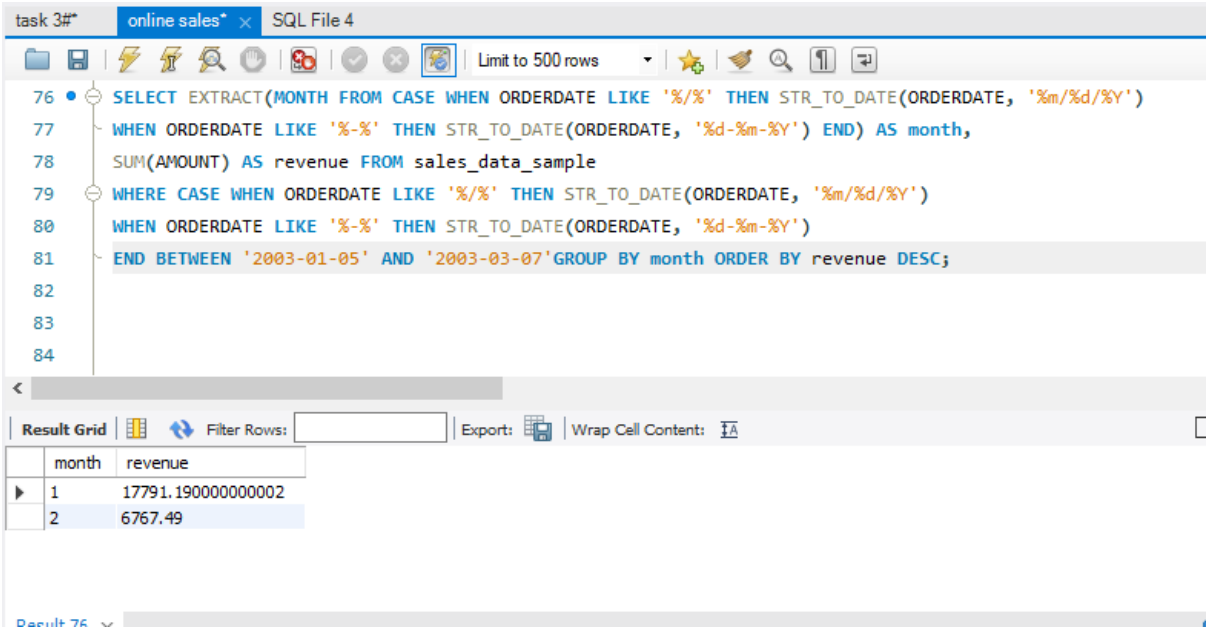
Result 77

10. FILTER (SPECIFIC TIME PERIOD)

- Restricts analysis to a date range.
- Example: Q1 (Jan–Mar) performance.
- This is how you do quarterly reporting.

```
SELECT
    EXTRACT(MONTH FROM
        CASE
            WHEN ORDERDATE LIKE '%/%' THEN
STR_TO_DATE(ORDERDATE, '%m/%d/%Y')
            WHEN ORDERDATE LIKE '%-%' THEN
STR_TO_DATE(ORDERDATE, '%d-%m-%Y')
        END
    ) AS month,
    SUM(amount) AS revenue
```

```
FROM sales_data_sample
WHERE (
    CASE
        WHEN ORDERDATE LIKE '%/%' THEN STR_TO_DATE(ORDERDATE, '%m/%d/%Y')
        WHEN ORDERDATE LIKE '%-%' THEN STR_TO_DATE(ORDERDATE, '%d-%m-%Y')
    END
  ) BETWEEN '2024-01-01' AND '2024-03-31'
GROUP BY month;
```



Full table:

Result Grid | Filter Rows:

month	revenue
2	34334.88
7	22127.18
1	37230.079999999994
8	37109.490000000005
10	54440.14
11	61460.00999999999
5	47189.31
6	23844.22
9	19764.71

Result 79

11. NULL HANDLING

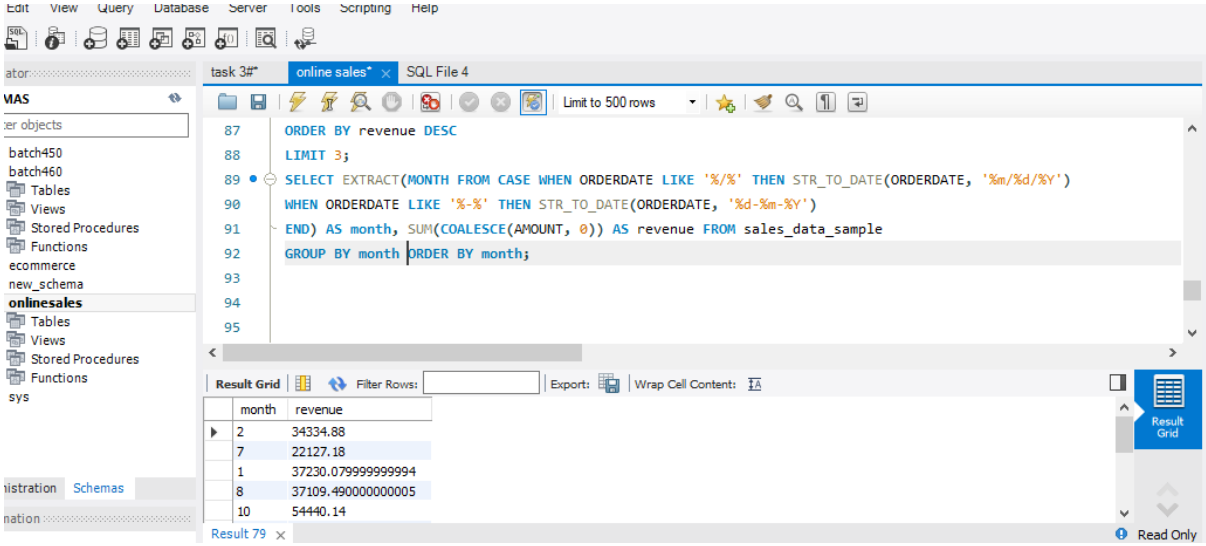
- Real data often has missing values.
- COALESCE(amount, 0) ensures calculations don't break.
- Without this, SUM might ignore NULLs or throw errors.
- This is data cleaning inside SQL.

```
SELECT
    EXTRACT(MONTH FROM
        CASE
            WHEN ORDERDATE LIKE '%/%' THEN
STR_TO_DATE(ORDERDATE, '%m/%d/%Y')
            WHEN ORDERDATE LIKE '%-%' THEN
STR_TO_DATE(ORDERDATE, '%d-%m-%Y')
        END
    ) AS month,
```

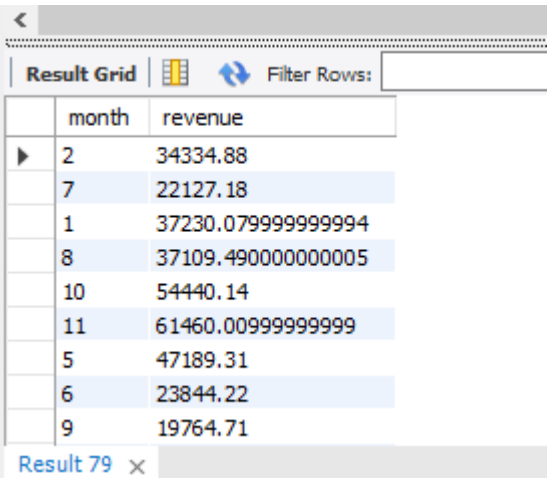
SUM(COALESCE(amount, 0)) AS revenue

FROM sales_data_sample

GROUP BY month;



Full table:



Result Grid | Filter Rows:

month	revenue
2	34334.88
7	22127.18
1	37230.079999999994
8	37109.490000000005
10	54440.14
11	61460.00999999999
5	47189.31
6	23844.22
9	19764.71

Result 79